

**LAPORAN RESMI MATA KULIAH WORKSHOP APLIKASI
BERBASIS WEB LAPORAN CRUD WITH LARAVEL**



Nama : Muhammad Yoga Ananda Satria

NRP : 3124500036

Dosen Pengajar : Mu'arifin S.ST., M.T

**PROGRAM STUDI D3 TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)
TAHUN 2025**

A. Tujuan Pembelajaran

1. Memahami konsep

CRUD (*Create, Read, Update, Delete*) dalam pengembangan aplikasi berbasis *web*.

2. Mampu mengimplementasikan fungsionalitas CRUD menggunakan *framework Laravel*.

3. Mampu menyiapkan dan menggunakan *master layout (Blade Template)* untuk konsistensi tampilan aplikasi.

4. Mampu menampilkan dan mengelola data pegawai (*employee*) melalui antarmuka *web*.

B. Dasar Teori

1. **CRUD** (*Create, Read, Update, Delete*): Konsep dasar dalam pengelolaan data pada basis data.

- **Create**: Operasi untuk membuat data baru (misalnya, membuat data pegawai baru).
- **Read**: Operasi untuk menampilkan atau membaca data (misalnya, menampilkan daftar pegawai).
- **Update**: Operasi untuk mengubah data yang sudah ada (misalnya, mengedit data pegawai).
- **Delete**: Operasi untuk menghapus data (misalnya, menghapus data pegawai).

2. **Laravel**: *Framework* PHP yang menggunakan arsitektur **MVC** (*Model-View-Controller*) untuk membangun aplikasi *web* yang kompleks.

3. **Blade Template Engine**: *Templating engine* yang digunakan Laravel untuk mendefinisikan *view*. Fitur utamanya adalah

@extends (untuk mewarisi *layout* dari *file* lain), **@yield** (untuk mendefinisikan bagian yang akan diisi), dan **@section** (untuk mengisi bagian yang di-*yield*).

C. Tugas

1. Kerjakan Percobaan di atas
2. Lakukan commit pada lokal repository anda pada branch dev dengan perintah git add .

selanjutnya git commit -m "Penambahan Master Layout"

3. Push ke remote repository anda. Pastikan sudah di branch dev lalu jalankan perintah push

origin dev

D. Jawaban

Menyiapkan Template

1. Membuat file master.blade.php sebagai master layout, dalam folder employees

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>@yield(section: 'title', default: 'App Pegawai')</title>
</head>

<body>
  <header>
    <h1>App Pegawai</h1>
    <!--<h1>@yield('page-title', 'App Pegawai')</h1> Kode ini tidak bekerja semestinya jadi saya komen agar tidak muncul!-->
    <nav>
      <ul>
        <li><a href="{{ url(path: '/employees') }}"> Employee</a></li>
        <li><a href="{{ url(path: '/department') }}">Department</a></li>
        <li><a href="{{ url(path: '/attendance') }}">Attendance</a></li>
        <li><a href="{{ url(path: '/report') }}">Report</a></li>
        <li><a href="{{ url(path: '/settings') }}">Settings</a></li>
      </ul>
    </nav>
  </header>
  <main>
    @yield(section: 'content')
  </main>
  <footer>
    <p>&copy; {{ date(format: 'Y') }} App Pegawai</p>
  </footer>
</body>

</html>
```

Penjelasan:

master.blade.php berfungsi sebagai **Master Layout** atau *template* utama aplikasi. Tujuannya adalah memastikan semua halaman memiliki struktur dasar dan header/footer yang seragam.

1. **Definisi Struktur Halaman:** File ini membuat kerangka HTML dasar, termasuk `<html>`, `<head>`, dan `<body>`.

2. **Penggunaan @yield:**

- `@yield(section: "title")`: Digunakan di `<head>` untuk *title* halaman.

Default judulnya adalah 'App Pegawai'.

- `@yield(section: "content")`: Digunakan di dalam `<main>` untuk menampung seluruh konten spesifik dari halaman turunan (seperti `index.blade.php`).

3. **Navigasi:** Element `<nav>` berisi daftar tautan (*link*) menggunakan *function* **route()** yang mengarah ke berbagai modul, seperti *Employee*, *Department*, *Attendance*, *Report*, dan *Settings*.

4. **Footer:** Menampilkan *copyright* tahun saat ini (`{{ date(format: "Y") }}`) dan nama aplikasi "App Pegawai".

2. menyesuaikan tampilan halaman utama employee pada file index.blade.php

```
@extends(view: 'employees.master')
@section(section: 'title', content: 'Daftar Pegawai')
@section(section: 'content')

<div class="container mt-5">
  <h1 class="mb-4">Daftar Pegawai</h1>
  <table border="1" cellpadding="5" cellspacing="0">
    <thead>
      <tr>
        <th>Nama Lengkap</th>
        <th>Email</th>
        <th>Nomor Telepon</th>
        <th>Tanggal Lahir</th>
        <th>Alamat</th>
        <th>Tanggal Masuk</th>
        <th>Status</th>
        <th>Aksi</th>
      </tr>
    </thead>
    <tbody>
      @foreach($employees as $employee)
        <tr>
          <td>{{ $employee->nama_lengkap }}</td>
          <td>{{ $employee->email }}</td>
          <td>{{ $employee->nomor_telepon }}</td>
          <td>{{ $employee->tanggal_lahir }}</td>
          <td>{{ $employee->alamat }}</td>
          <td>{{ $employee->tanggal_masuk }}</td>
          <td>{{ $employee->status }}</td>
          <td>
            <a href="{{ route(name: 'employees.show', parameters: $employee->id) }}">Detail</a> |
            <a href="{{ route(name: 'employees.edit', parameters: $employee->id) }}">Edit</a> |
            <form action="{{ route(name: 'employees.destroy', parameters: $employee->id) }}" method="POST" style="display:inline;">
              @csrf
              @method('DELETE')
              <button type="submit" onclick="return confirm('Yakin ingin menghapus?')">Delete</button>
            </form>
          </td>
        </tr>
      @endforeach
    </tbody>
  </table>
</div>
@endsection
```

Penjelasan:

index.blade.php adalah *view* yang bertanggung jawab menampilkan **daftar data pegawai** dan mengimplementasikan operasi **Read** dari CRUD.

1. **Mewarisi Layout:** Menggunakan `@extends('view: "employees.master")` untuk mewarisi struktur dari `master.blade.php`.
2. **Mengisi Konten:** Menggunakan `@section('section: "content")` untuk mengisi area `@yield("content")` pada *master layout*.
3. **Menampilkan Data:** Data ditampilkan dalam sebuah tabel.

- Digunakan perulangan

@foreach (\$employees as \$employee) untuk mengambil dan menampilkan setiap data pegawai (\$employee) ke dalam baris tabel (<tr>).

4. **Aksi (CRUD):** Kolom **Aksi** menyediakan fungsionalitas:

- **Detail:** Tautan ke *route* employees.show untuk operasi **Read** data tunggal.
- **Edit:** Tautan ke *route* employees.edit untuk operasi **Update**.
- **Delete:** Menggunakan elemen <form> dengan **@method('DELETE')** untuk melakukan operasi **Delete**. Tombol ini akan memunculkan konfirmasi "Yakin ingin menghapus?" sebelum eksekusi.

Output:

App Pegawai

- [Employee](#)
- [Department](#)
- [Attendance](#)
- [Report](#)
- [Settings](#)

Daftar Pegawai

Nama Lengkap	Email	Nomor Telepon	Tanggal Lahir	Alamat	Tanggal Masuk	Status	Aksi
M.Yoga Ananda Satria	jumbo@admin.com	085804709019	2025-09-28	dfghjk	2025-09-29	aktif	Detail Edit Delete

E. Analisa

1. **Penerapan MVC dan Blade:** Implementasi ini berhasil memisahkan *logic* tampilan (*View*) dari *logic* bisnis menggunakan Blade Template. Penggunaan `master.blade.php` dan `index.blade.php` menunjukkan penerapan pola **Don't Repeat Yourself (DRY)**, di mana elemen umum aplikasi tidak perlu ditulis berulang kali.
2. **Implementasi CRUD Read dan Link:** Data pegawai berhasil ditampilkan dalam bentuk tabel (**Read**). Kolom aksi berhasil menyediakan *link* untuk operasi **Detail**, **Edit**, dan **Delete**, yang menunjukkan kesiapan *view* untuk berinteraksi dengan operasi CRUD lainnya.
3. **Metode DELETE yang Benar:** Penggunaan `<form>` dengan `@method('DELETE')` untuk operasi penghapusan adalah praktik yang benar dalam Laravel. Hal ini memastikan bahwa operasi penghapusan dilakukan melalui *request* HTTP DELETE, bukan GET atau POST biasa.
4. **Tampilan Output:** *Output* menunjukkan bahwa *layout* master dan konten index berhasil digabungkan, menampilkan *header* navigasi, dan tabel data pegawai dengan lengkap dan fungsional.

F. Kesimpulan

1. Proyek

CRUD with Laravel telah berhasil diselesaikan dengan fokus pada persiapan *template* dan implementasi tampilan daftar data (**Read** data) pegawai.

2. **Blade Templating** (melalui `master.blade.php` dan `index.blade.php`) berhasil digunakan untuk menciptakan antarmuka pengguna yang terstruktur dan konsisten.

3. Fungsionalitas dasar untuk melihat data pegawai, serta

link untuk **Detail**, **Edit**, dan **Delete** sudah siap diakses, menunjukkan fondasi yang kuat untuk menyelesaikan seluruh operasi CRUD.

4. Semua kode yang dibuat sudah diunggah dan tersedia di

GitHub pada *link* yang telah disediakan.

G.Link Github

<https://github.com/Rapprince29/Laravel-11>